

PHASE III: REFLECTION

AIDAN SHEEHY, YAROSLAV TRACH, MURAT YILDIZ

CSC-220 OPERATING SYSTEMS & SYSTEMS PROGRAMMING

DR.KANCHARLA

THURSDAY APRIL 30TH, 2026

1. How was non-blocking input implemented?

Non-blocking input was implemented in a dedicated `input_thread`. The program first changes the terminal into non-canonical mode and disables echo using `termios`, so the user can press single keys without needing Enter. It also sets `STDIN_FILENO` to `O_NONBLOCK` with `fcntl`, so keyboard reading does not freeze the program. The input thread uses `select()` with a short timeout, then reads available characters with `read()` and sends each character to `handle_char()`. This lets the dashboard keep refreshing while input is checked in the background.

2. How does the input thread interact with other subsystems?

The input thread interacts with the rest of the system by updating shared `g_state` fields through mutex-protected sections. For example, W adds acceleration steps, S adds deceleration steps, C enables cruise mode, A and D toggle turn signals, Z toggles hazard lights, and H toggles the headlight. Engine commands are passed through helper functions like `system_engine_kill()` and `system_engine_ignite()`. After ECU-related changes, the input thread calls `sync_notify_ecu()` so the ECU thread wakes up and recalculates derived state like signal status, RPM zone, and temperature classification.

3. How does the ECU handle invalid user actions?

The ECU enforces safety rules after input changes are made. If the engine is off, it forces speed and RPM back to zero and clears acceleration, deceleration, and cruise mode. If refueling is active while the motorcycle is moving, the ECU cancels refueling. If the fuel is empty or the system requires refueling before start, the ECU keeps the engine off and clears movement-related state. It also limits RPM and speed during overheating and limits speed when fuel is low. These rules prevent invalid user actions from putting the simulation into an unsafe or unrealistic state.

4. What challenges were encountered when integrating input with synchronization?

The main challenge was making user input responsive without creating race conditions in shared state. Since the input thread, engine thread, motion thread, fuel thread, ECU thread, and dashboard thread can all access `g_state`, updates had to be protected with the correct mutexes. Another challenge was avoiding deadlocks, so shared data is locked only when needed and ECU updates use condition variables instead of constant busy waiting. The ECU also uses a timed wait on `cond_ecu`, which allows it to wake quickly when input changes happen but still check state regularly if no signal arrives.

5. How was graceful shutdown implemented?

Graceful shutdown was implemented with a global `g_shutdown_request` flag. Pressing Q in the input thread sets this flag, and pressing Ctrl+C also sets it through the signal handler. The main thread waits until the shutdown flag becomes true, then calls `wake_all_blocked_threads()`

to broadcast condition variables and wake any blocked threads. After that, it joins the input, dashboard, ECU, fuel, motion, and engine threads, restores the terminal settings, closes the log, destroys mutexes and condition variables, and exits cleanly.